Recognize Chinese handwritten numbers using MLP networks

Student: Iulia Onică

Discipline: Machine Learning

1. Multilayer perceptron (MLP)

Multi-layer perceptron (MLP) is a supplement of feed forward neural network. It consists of three types of layers—the input layer, output layer and hidden layer (Figure 1.)

The **input layer** receives the input signal to be processed. The required task such as prediction and classification is performed by the output layer. An arbitrary number of **hidden layers** that are placed in between the input and output layer are the true computational engine of the MLP. Similar to a feed forward network in a MLP the data flows in the forward direction from input to **output layer**.

The neurons in the MLP are trained with the back propagation learning algorithm. MLPs are designed to approximate any continuous function and can solve problems which are not linearly separable. The major use cases of MLP are pattern classification, recognition, prediction and approximation.

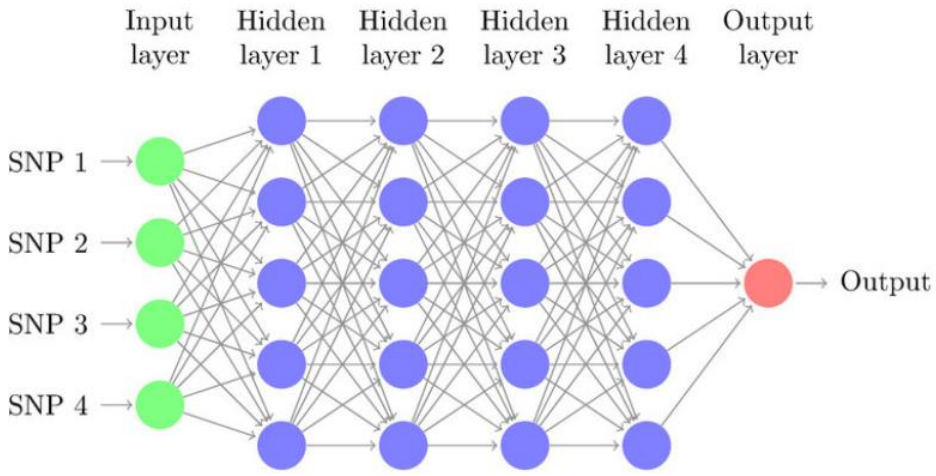
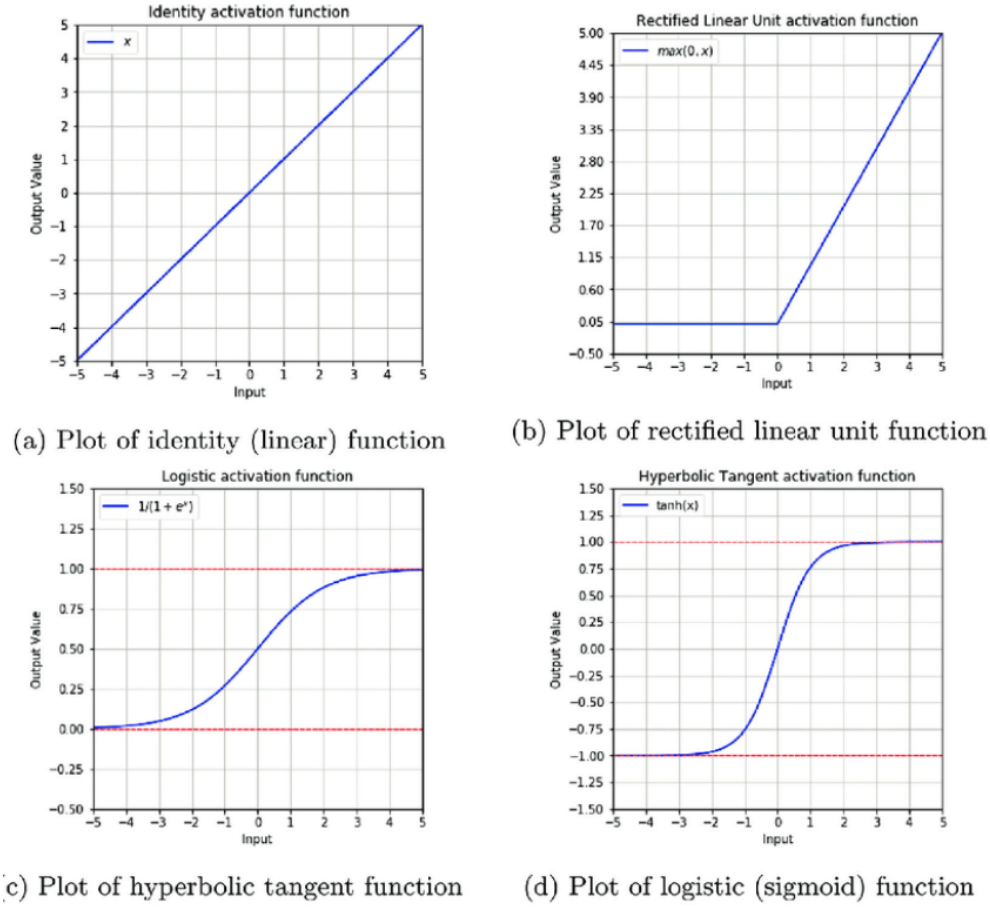


Figure 1. Multilayer perceptron

After the MLP layer has been designed the next step is to back propagate to each layer and modify weights to reduce the mean squared difference between the expected output and predicted output. We do this by first calculating the loss and then using an optimizer that will optimize the weights to reduce loss.

1.1. Activation Function

At the level of one perceptron, the input layer acts as the dendrites and is responsible for receiving the inputs. The number of nodes in the input layer is equal to the number of features in the input dataset. Each input is multiplied with a weight (which is typically initialized with some random value) and the results are added together (Figure 3). The sum is then passed through an **activation function**.



(a) Plot of identity (linear) function

(b) Plot of rectified linear unit function

(c) Plot of hyperbolic tangent function

(d) Plot of logistic (sigmoid) function

Figure 2. Most common activation functions

The activation function of a perceptron resembles the nucleus of human nervous system neuron. It processes the information and yields an output. In the case of a perceptron, this output is the final outcome. However, in the case of multilayer perceptrons, the output from the neurons in the previous layer serves as the input to the neurons of the proceeding layer.

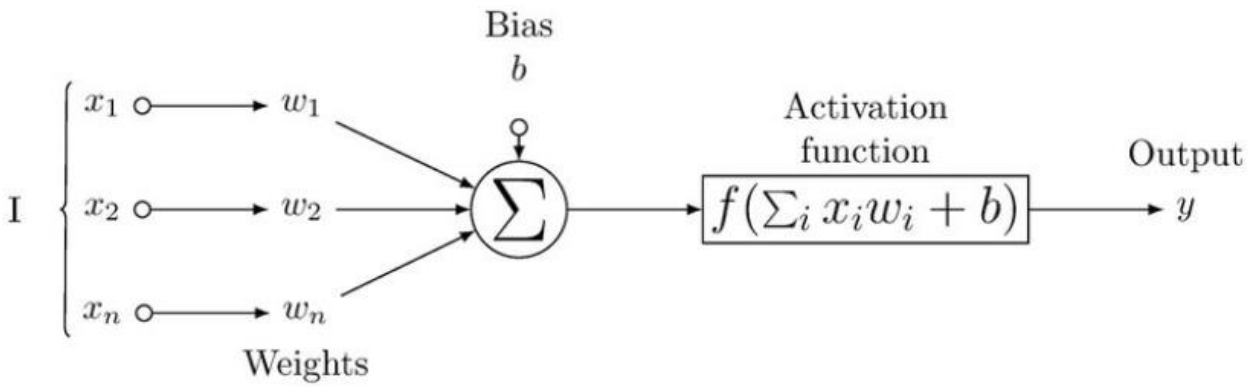


Figure 3. Perceptron's parameters

1.2 Cost function

In practicality most neural networks tend to use **cross entropy**. Many beginner's classes and tutorials on neural network will show mean squared error, as it is probably more intuitive and easier to understand at first.

The loss function is computed with the following formula:

$$L(y, \hat{y}) = -y_1 \log \hat{y_1} - \cdots - y_n \log \hat{y_n} = -\log \hat{y_{class}}$$

The loss function was defined with respect to a single training example. It measures how well it is doing on a single training example.

The cost function measures how well it is doing an entire training set. Thus, the cost function J which is applied to the parameters W and B is going to be the average with one of the m of the sum of the loss function applied to each of the training examples and turn.

$$J = \frac{1}{N} \sum_i L(y^i, \hat{y}^i)$$

2. Practical example using MNIST Chinese dataset

Download page: https://www.kaggle.com/gpreda/chinese-mnist

Using this dataset one can train his own model in order to classify the Chinese number characters.

This dataset provides the user with 15 000 images, each of them with 64x64 resolution. There are totally 100 suites, each created by a volunteer. Each volunteer created 10 samples. And each sample contains characters from 0 to 100M (totally 15 Chinese number characters).

The images have the following name:

- ✓ "input_suite_sample_number.jpg"

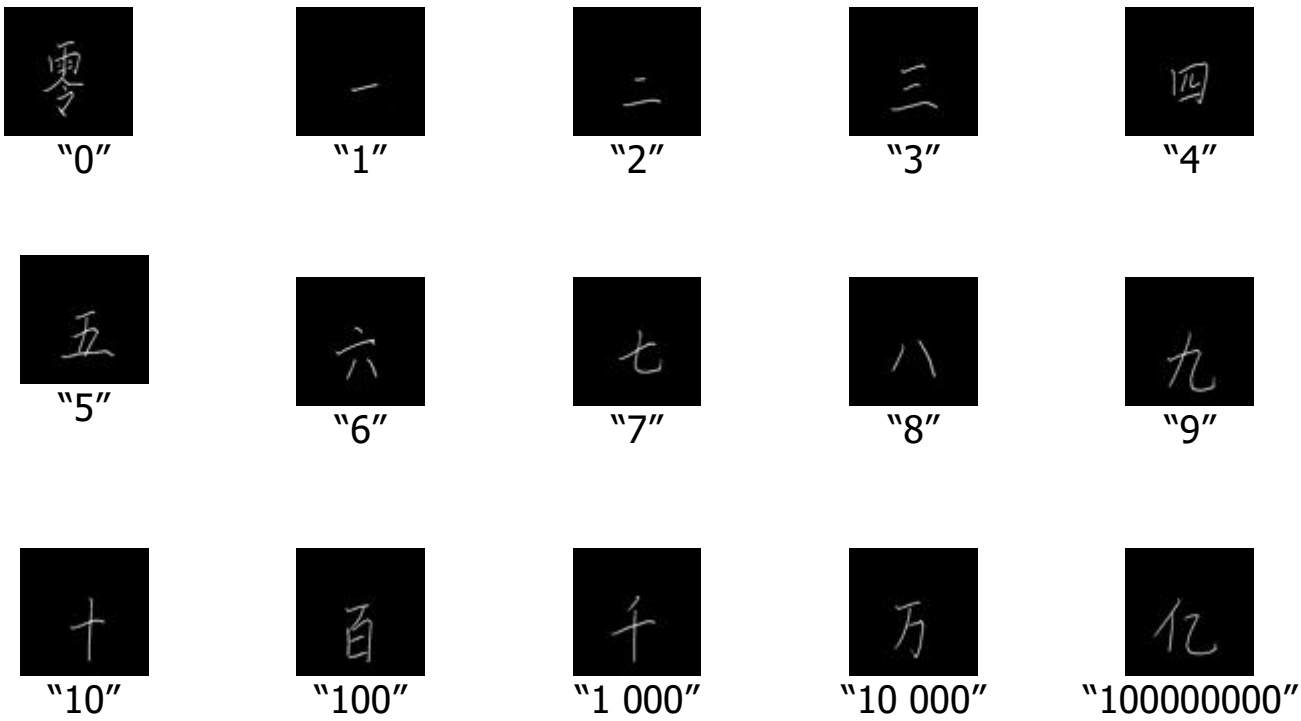


Figure 4. Chinese number characters

2.1 Data preprocessing

```
clear all;
close all;
cd 'C:\Users\iulia.onica\Desktop\Proiect_ML\Dataset_chinese\data\data';

X = [];
imagefiles = dir('*.jpg');
nfiles = length(imagefiles);    % Number of files found
S = [];
for j = 1 : 100
    for k = 1:10
        for l = 1:10
            S = strcat('input_',num2str(j),'_', num2str(k), '_',num2str(l),
'.jpg');
            for i = 1 : nfiles
                currentfilename = imagefiles(i).name;
                if strcmp(S,currentfilename) == 1
                   currentimage = imread(currentfilename);
                   currentimage = imresize(currentimage, [26 26]);
                   currentimage = im2double(currentimage);
                   array = reshape(currentimage.',1,[]);
                   array = [l array]
                   X =[X; array];
                   S = [];
                end;
            end;
        end;
    end;
end;

random_X = X(randperm(size(X, 1)), :);
csvwrite('chinese_MNIST_train.csv', random_X);
```

In the above code sequence, the images are read from **data** directory and put as row arrays in the matrix **X**. Because the computer required some memory constrains, there will be processed only 10 classes out of 15. This means, the classification will be done only for the digits' characters. Also, the images will be resized from 64x64 to 56x56, 30x30, 28x28 and finally 26x26 pixels. This is done because of the **"out of memory" error** but also for performance testing purposes.

Each array has attached at the beginning an integer that represents **the corresponding Chinese number**. The rows of the output matrix X are being randomized in order to prevent some overfitting.

Overfitting means that the model learns too much from training data and isn't able to generalize the underlying information. When this happens, the model is able to describe training data very accurately but loses precision on every dataset it has not been trained on.

The variable X is written in a **.csv** file to make it easier at the training part. The user will just load it, not being necessary to create it all over again.

```
close all;
clear all;
cd 'C:\Users\iulia.onica\Desktop\Proiect_ML\Dataset_chinese\data';
load chinese_MNIST_train.csv

X = chinese_MNIST_train(:, 2:end); %input data
Y = chinese_MNIST_train(:,1); %output data

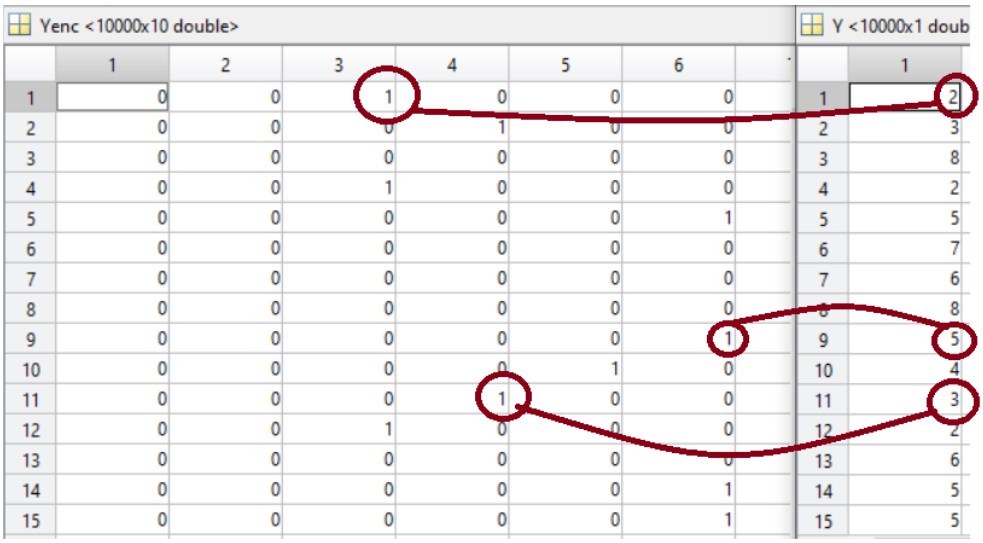
Yenc = full(ind2vec(1 + Y')); % 10 classes from 1 to 10
Yenc = Yenc';                 % same dimension as input data

network = patternnet([100, 70]);     % 2 hidden layers of 100 and 70 neurons
network = train(network, X',Yenc');  % Train the network with the data
```

In this new script file, the input file X will contain the images pixels as rows and Y will have the corresponding number character.

One hot encoding allows the representation of categorical data to be more expressive. Many machine learning algorithms cannot work with categorical data directly. The categories must be converted into numbers. This is required for both input and output variables that are categorical.

Thus, we obtain Yenc of 10000x10 dimension. There are 10 000 images for the 10 classes, and for each of them in Yenc is stored on the rows an "1" at the position n+1, where n is the corresponding Chinese number.



	1	2	3	4	5	6		1
1	0	0	1	0	0	0	1	2
2	0	0	0	1	0	0	2	3
3	0	0	0	0	0	0	3	8
4	0	0	1	0	0	0	4	2
5	0	0	0	0	0	1	5	5
6	0	0	0	0	0	0	6	7
7	0	0	0	0	0	0	7	6
8	0	0	0	0	0	0	8	8
9	0	0	0	0	0	1	9	5
10	0	0	0	0	1	0	10	4
11	0	0	0	1	0	0	11	3
12	0	0	1	0	0	0	12	2
13	0	0	0	0	0	0	13	6
14	0	0	0	0	0	1	14	5
15	0	0	0	0	0	1	15	5

Figure 5. One hot encoding performed on output column vector

2.2 Analysis over performance

2.1.1 All 15 classes with 56x56 image resolution

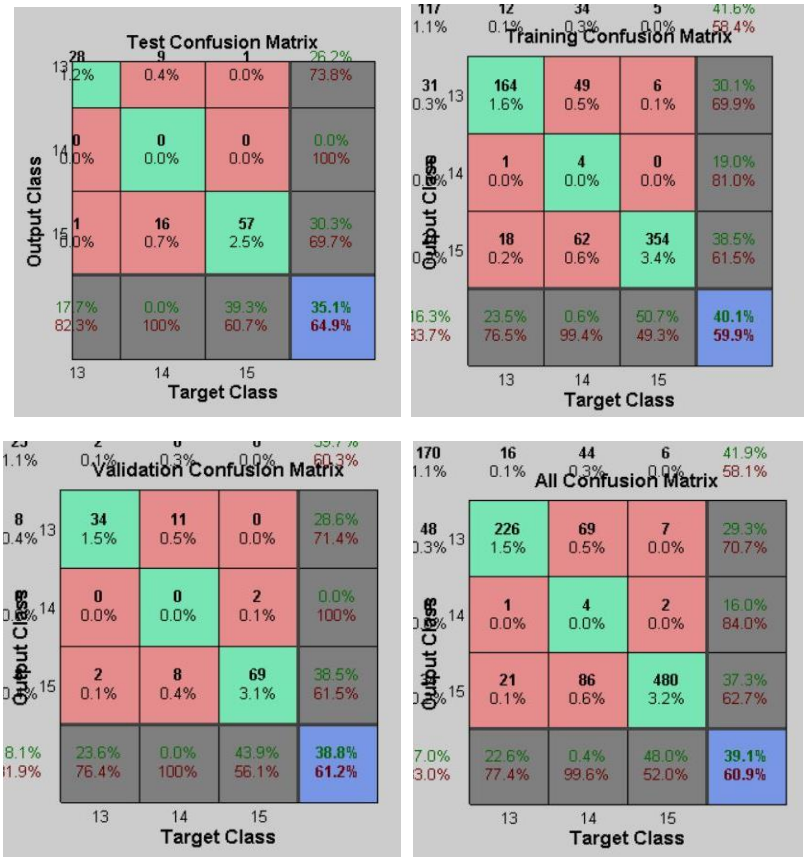
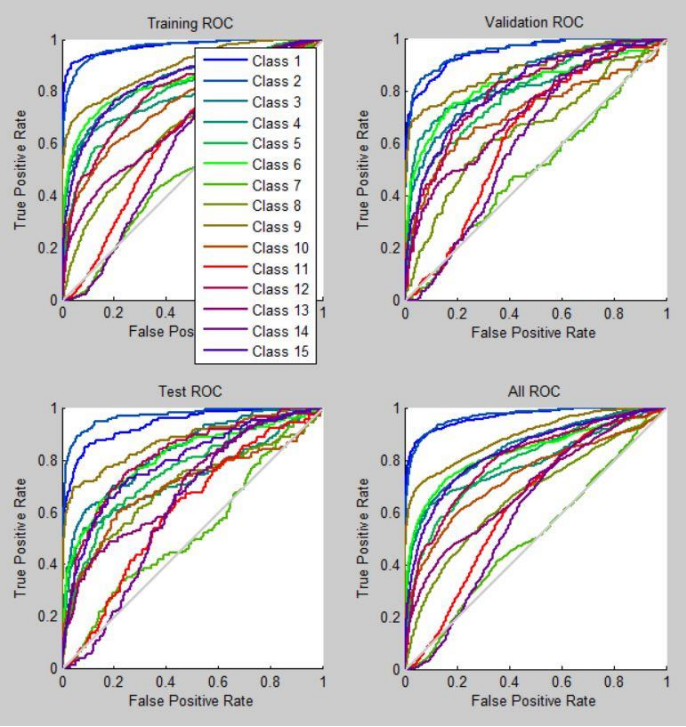


Figure 6. Confusion matrix for 15 classes

As there can be seen, the results are pretty **bad**. Basically, the model can not distinguish at all the numbers. The cell in the bottom right of the plot shows the overall accuracy. Generally, the worst case scenario is when it is around 50% when it can not tell which is correctly classified and which not.

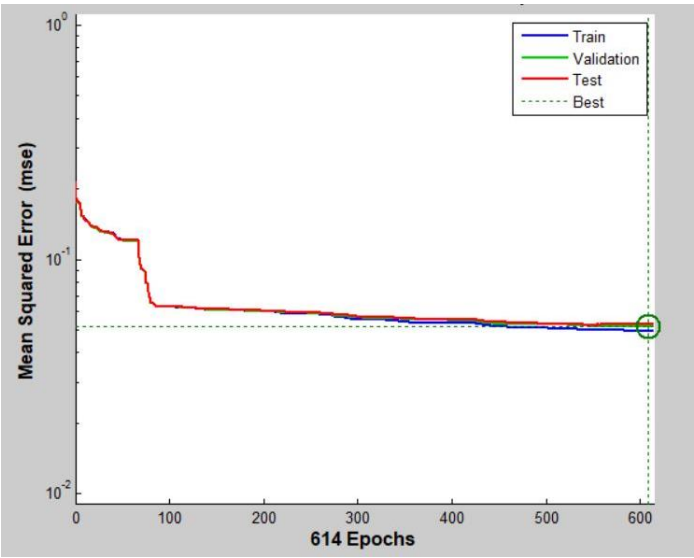
Because there is an equal number of images for each class, the diagonal should have values around 6,6%, ideally (representing 10 000/15 = 666.6 images out of 10 000).

The **receiver operating characteristic** is a metric used to check the quality of classifiers. For each class of a classifier, ROC applies threshold values across the interval [0,1] to outputs. For each threshold, two values are calculated, the True Positive Ratio (TPR) and the False Positive Ratio (FPR).



The Chinese numbers 0 and 1have a higher probability of correct estimation than 6 or 7.

Figure 7. Receiver operating characteristic



The best validation performance is 0.051624 at epoch 608.

Figure 8. Mean Squared Error

2.1.2 Using 10 classes with 30x30 image resolution

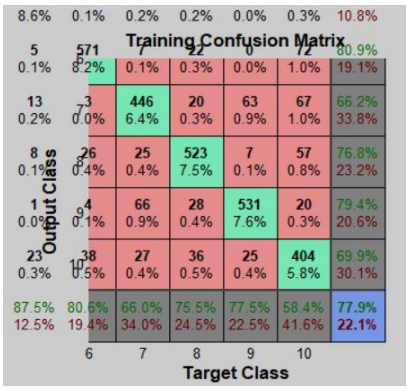


Figure 9. Two hidden layers with 60 neurons [training]

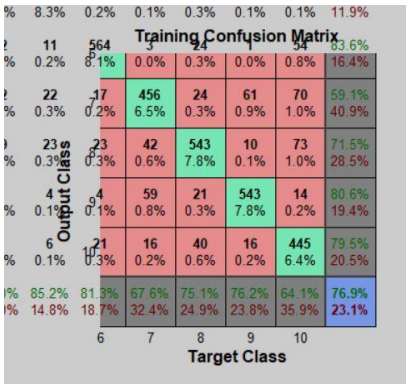


Figure 10. Two hidden layers with [80 60] neurons

Performance has visibly increased.

2.1.3 Using 10 classes with 28x28 image resolution

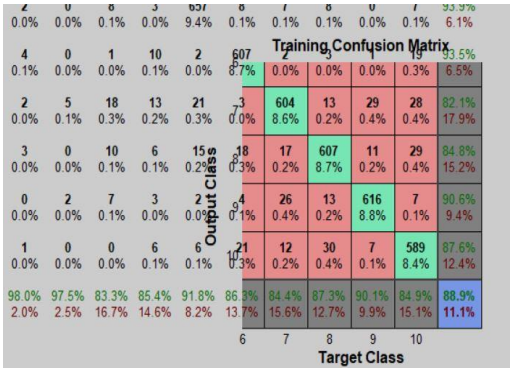


Figure 11. Two hidden layers with [90 70]

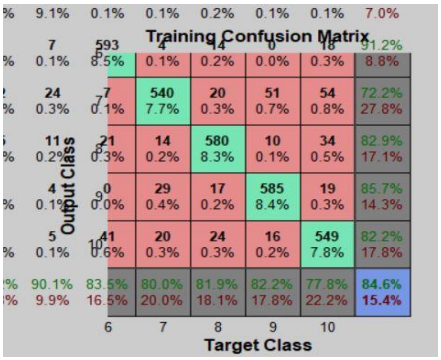


Figure 12. Two hidden layers with [80 60] neurons

Best accuracy obtained until now: 88.9%.

2.1.4 Using 10 classes with 26x26 image resolution

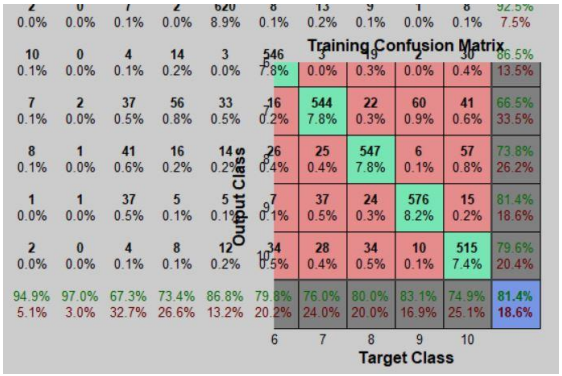


Figure 13. Two hidden layers with [40 40] neurons

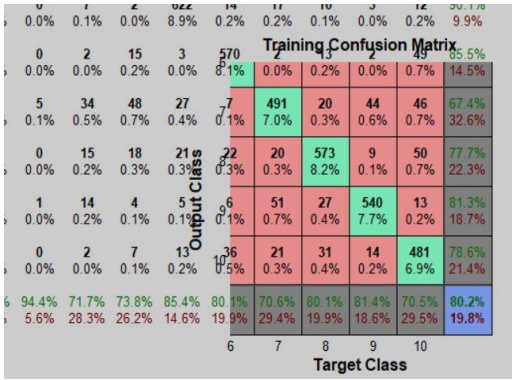


Figure 14. Two hidden layers with [40 60] neurons

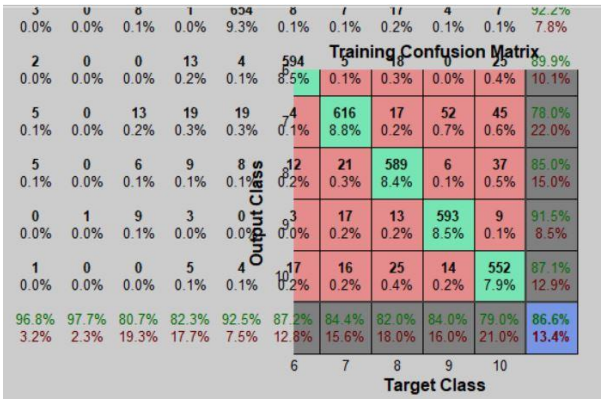


Figure 15. Two hidden layers with [60 40] neurons

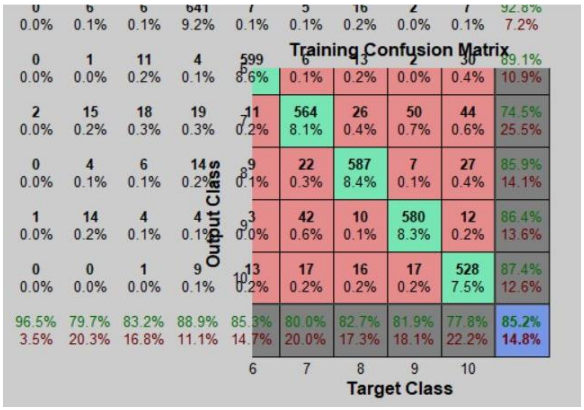


Figure 16. Two hidden layers with [60 60] neurons

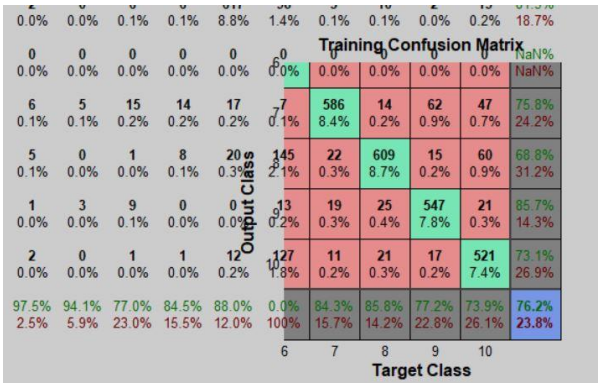


Figure 17. Two hidden layers with [60 80] neurons

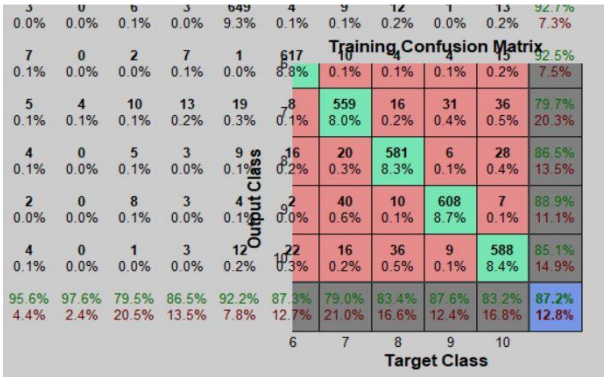


Figure 18. Two hidden layers with [80 80] neurons

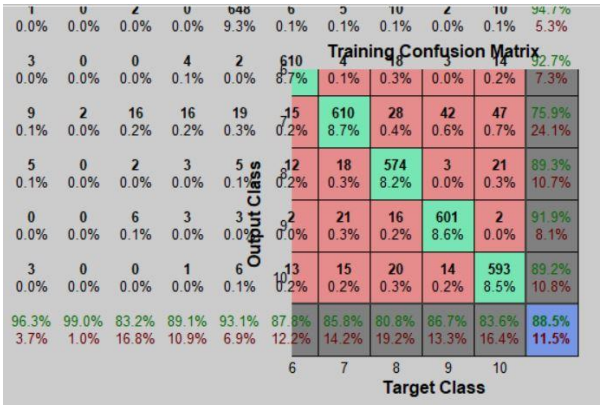


Figure 19. Two hidden layers with [100 80] neurons

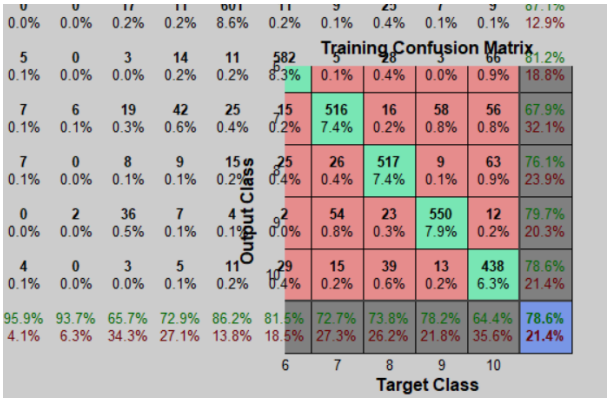


Figure 20. Three hidden layers with [90 70 60] neurons

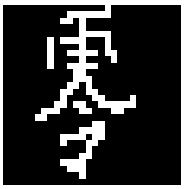
OBS: A higher number of hidden layers does not seem to increase the accuracy. Also, neither one hidden layer was proved to be useful.

(Observation are made based on testing).

2.3 Testing handwritten Chinese letters

According to the ROC representation 0 and 1 are classified the best. There will be used the network that has the best performance (2 hidden layers of 90, 70 neurons).

Here is a picture in Paint of "0", 28x28 pixels:



Using the following line codes, we can verify the accuracy of the network:

```
%test network
I = imread('zero.jpg');
I = im2double(rgb2gray(I));

%Convert to line
I = I';
Ivec = I(:);

%Inference
outputs = network(Ivec);
```

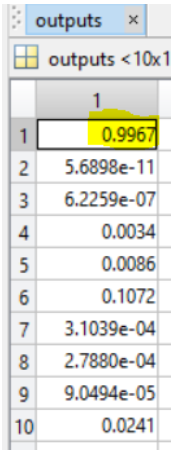


Figure 21. Output for "0" input

The picture is transformed in an array of pixels and given to the network to be interpreted. The output will display the probabilities of being a "0" for all the classes. The one closer to 1 is practically the estimation given by the model. As there can be seen the model sees it indeed as an "0", with a probability of 99.67%.

Not every time the numbers will be correctly estimated. For instance, a drawn "7" outputs a probability of 7.01% (Figure 22), which is not so good.

In fact, there are several factors that influence the output: the image has only white and black (but the model was trained with all kinds of gray

shades), the dimension of the character written on the black background (most of them are centered in the middle and smaller), the thickness of the character (the dataset has the characters much thinner) or the character has been written wrongly (unknown as they are Chinese characters).

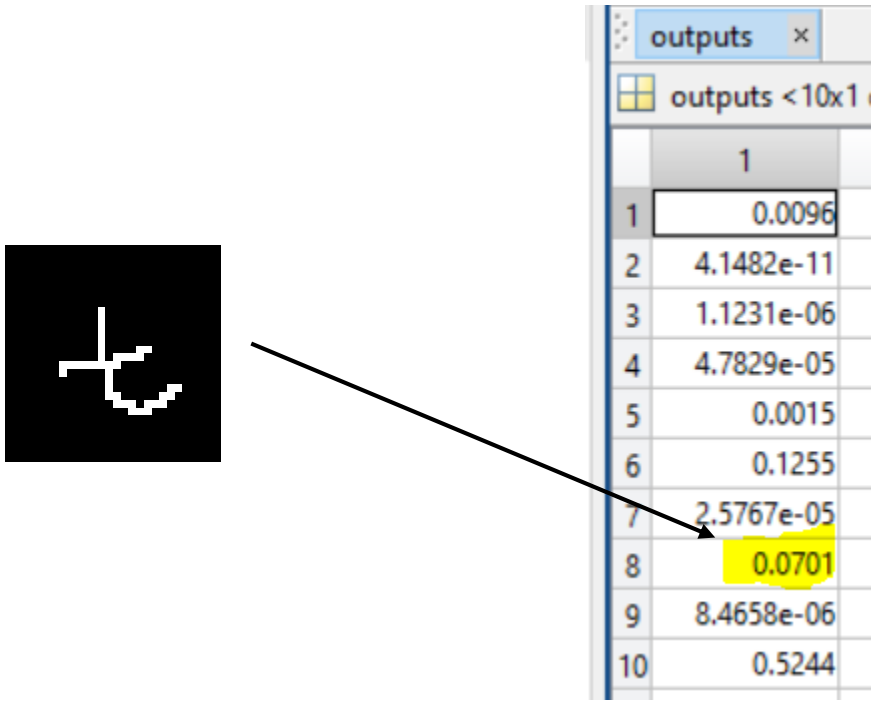


Figure 22. Output for "7" input